Computer Networks Lab
Fall 2025
Week 06

Socket Programming (UDP)

Lab Task

Task 1: Multi-Client Echo Server with Message Counter

You work for a network monitoring company that needs to test UDP communication reliability. Design a UDP echo server that can handle multiple clients simultaneously and keep track of message statistics.

Server Implementation:

- Create a UDP server that listens on port **8080**
- The server should run in an **infinite loop** to handle multiple client requests
- For each received message, the server should:
  - Print the received message
  - Maintain a **message counter** (starting from 1)
  - Send back an echo response in the format: "Echo #[counter]: [original_message]"
  - Display total messages received so far

Client Implementation:

- Create a UDP client that connects to the server on localhost:8080
- The client should:
  - Send 3 different messages to the server (e.g., "Hello Server", "How are you?", "Goodbye")
  - Wait for and display each echo response
  - Use a small delay (1-2 seconds) between messages

Expected Output Example:

Server Output:

```
UDP Echo Server listening on port 8080...
Waiting for clients...
```

```
Received message: "Hello Server"
Total messages: 1

Received message: "How are you?"
Total messages: 2

Received message: "Goodbye"
Total messages: 3

Received message: "Hello Server"
Total messages: 4

Received message: "How are you?"
Total messages: 5

Received message: "Goodbye"
Total messages: 6
```

Client Output:

```
=== UDP Echo Client ===
Connecting to server at localhost:8080

Sending: Hello Server
Server response: Echo #1: Hello Server

Sending: How are you?
Server response: Echo #2: How are you?

Sending: Goodbye
Server response: Echo #3: Goodbye

Client finished.
```

Task 2: Simple Student Grade System

You're developing a basic student grade lookup system for a university. Students can query their grades from a server using their student ID. The server has a simple database of student records.

Server Implementation:

- Create a UDP server that listens on port **9001**

- Initialize a **simple student database** with 5 student records:
  - Student ID (e.g., "2021-CS-001")
  - Student Name
  - Course Grade (e.g., "A", "B+", "F")
- Server should handle student ID queries:
  - Receive student ID from client
  - Look up student in database
  - Return student name and grade
  - If student ID not found, return "Student not found"

Client Implementation:

- Create an interactive UDP client that:
  - Prompts user to enter student ID
  - Sends student ID to server
  - Displays the server response
  - Continues until user types "EXIT"
  - Shows simple help instructions

Sample Student Database:

2021-CS-001	Alice Johnson	A
2021-CS-002	Bob Smith	F
2021-CS-003	Carol Davis	A-
2021-EE-001	David Wilson	B
2021-EE-002	Eve Brown	C+

Expected Output Example:

Server Output:

```
=== Student Grade Server ===
Server listening on port 9001...
Database has 5 students

Student ID query: "2021-CS-001"
Result: Name: Alice Johnson, Grade: A

Student ID query: "2021-EE-001"
Result: Name: David Wilson, Grade: B

Student ID query: "2021-XX-999"
Result: Student not found
```

Client Output:

```
=== Student Grade System ===
```

```
Instructions:
- Enter student ID (e.g., 2021-CS-001)
- Server will return name and grade
- Type 'EXIT' to quit
================================

Enter Student ID: 2021-CS-001
Result: Name: Alice Johnson, Grade: A

Enter Student ID: 2021-EE-001
Result: Name: David Wilson, Grade: B

Enter Student ID: 2021-XX-999
Result: Student not found

Enter Student ID: EXIT
Goodbye!
```